

Backtesting

Architectures, Cost Modelling, Bias Controls, and Validation Methodology

Testing Whether a Strategy Would Have Made Money — and Whether That Result Can Be Trusted

A Level 3 module in the Quantitative Finance Curriculum

Covers: vectorised vs event-driven backtesting · signal/position generation · portfolio accounting · transaction costs/slippage/market impact/bid-ask modelling/latency/funding/borrow costs/partial fills · look-ahead bias/survivorship bias/selection bias/data snooping/overfitting/leakage · in-sample vs out-of-sample · walk-forward validation · purged cross-validation/embargo periods

Table of Contents

How to Use This Guide

What You Need to Know Before Starting

Understand Before Proceeding

Why It Matters

How This Connects to the Rest of the Curriculum

Glossary of Recurring Terms

Part 1 — Backtesting Architectures: From Signal to Ledger

1.1 Vectorised vs. Event-Driven Backtesting

1.2 Signal and Position Generation

1.3 Portfolio Accounting

Part 2 — Modelling the Real Cost of Trading

2.1 Transaction Costs, Slippage, and the Cost Waterfall

2.2 Bid-Ask Spread and Market-Impact Modelling

2.3 Latency, Funding, and Borrow Costs

2.4 Partial Fills

Part 3 — The Biases That Lie to You

3.1 Look-Ahead Bias

3.2 Survivorship Bias

3.3 Selection Bias and Data Snooping

3.4 Overfitting and Leakage

Part 4 — Validation Methodology

4.1 In-Sample vs. Out-of-Sample

4.2 Walk-Forward Validation

4.3 Purged Cross-Validation and Embargo Periods

Practical Exercise

Common Beginner Mistakes

Advanced Extensions

Professional Applications

Where to Go Next

How to Use This Guide

This document is a companion to **Backtesting**, the Level 3 module covering the methodology for testing whether a strategy would have made money — and, far more importantly, whether that result is trustworthy. It is organized around a simple observation: a backtest is a piece of software, and like any piece of software it can have bugs; the specific bugs that matter here are the ones that make a strategy look better than it actually is, because those are the ones nobody notices until real money is at risk.

Each section pairs a concept with a worked example that shows its effect quantitatively — how much a specific bias inflates a Sharpe ratio, how much a specific cost assumption changes a net return — because the whole point of this module is that these effects are measurable, not merely cautionary tales. Read start to finish before building your first backtesting engine, or jump to the section relevant to a bug you suspect — but do not skip **Understand Before Proceeding** below.

What You Need to Know Before Starting

- **Time Series** — specifically train/test splitting, and why data ordered in time cannot be shuffled the way i.i.d. data can.
- **Statistics** — specifically multiple testing and why evaluating many candidate strategies raises the bar for what counts as a genuine result, not just the mechanics of a single hypothesis test.
- **Trading Mechanics** — a working understanding of what transaction costs are (spread, commissions, financing) at a conceptual level, built on in the Market Microstructure module.

Understand Before Proceeding

***Prerequisite gate** — Do not trust any backtest result before this section — specifically look-ahead bias, survivorship bias, and transaction-cost modelling — is Depth 3+. A backtest that has not been deliberately audited for these three issues should be treated as an unverified claim, not a result. This is not a matter of being extra careful; it is the default epistemic status of any backtest you have not personally stress-tested against these failure modes, including backtests produced by experienced colleagues and published in reputable venues.*

Why It Matters

A backtest is only as good as its bias controls. Most 'strategies that stopped working' were never really working — the backtest was lying, due to look-ahead bias, survivorship bias, or overfitting. This reframing matters enormously in practice: the popular narrative of a strategy that 'used to work but got arbitrated away' implies a real edge that decayed, which is a very different (and, for the researcher, a much less uncomfortable) story than a backtest that never measured a real edge in the first place, because it was quietly using information that would not have been available at the time, or was evaluated on a universe that excluded every company that failed.

The stakes of getting this wrong are asymmetric in a specific way: a biased backtest does not fail loudly. It produces a strategy that looks excellent on paper, passes every review that does not specifically probe for these issues, gets allocated real capital, and then underperforms in a way that is very difficult to

diagnose after the fact — because by then, the live losing streak looks statistically unremarkable next to a fictional in-sample track record, rather than looking like what it actually is: the first honest measurement the strategy has ever received.

How This Connects to the Rest of the Curriculum

- **Research** — This is the validation gate every signal must pass before proceeding to portfolio construction; no strategy family from the Alpha Research module should be sized or combined into a portfolio until it has cleared the bias and validation checks in this module.
- **Development** — Building a correct, reusable, well-tested backtesting engine is itself a Level 4 development deliverable (see the Advanced Projects in Part VI) — the engine is infrastructure that every future strategy will depend on, so its correctness compounds across the entire research program.
- **Trading** — The gap between backtested and live performance is almost always explained by something in this section — a cost that was underestimated, a bias that was not controlled for, or a validation step that was skipped — rather than by 'the market changed.'

Glossary of Recurring Terms

Vectorised backtest — A backtesting approach that applies signal, position, and P&L calculations to entire arrays of historical data at once, using array/matrix operations rather than a step-by-step loop.

Event-driven backtest — A backtesting approach that processes one time step (or one market event) at a time through a pipeline of market-data, signal, order, and fill events, mirroring how a live trading system operates.

Slippage — The difference between the price assumed at the time a trading decision is made and the price actually achieved on simulated or real execution.

Market impact — The component of slippage caused by a trade's own size interacting with finite available liquidity.

Borrow cost — The fee paid to borrow shares in order to sell them short, which varies by security and can spike sharply for 'hard-to-borrow' names.

Funding rate — A periodic payment exchanged between long and short holders of certain instruments (notably perpetual futures) designed to tether the instrument's price to an underlying reference price.

Partial fill — An execution outcome in which only part of a submitted order size is actually traded, typically because available resting liquidity was smaller than the order.

Look-ahead bias — A backtesting error in which information that would not have been available at a given point in time is used in a signal or decision as of that point in time.

Survivorship bias — A backtesting error in which the historical universe of tradeable instruments is restricted to those that still exist today, silently excluding failures, delistings, and bankruptcies.

Selection bias — A backtesting error in which the sample of data, instruments, or time periods studied is not representative of the population the strategy will actually be traded on.

Data snooping — The practice, often inadvertent, of repeatedly testing variations of a strategy against the same dataset until one appears to work, inflating the apparent significance of the result.

Overfitting — A model or strategy that has fit the noise specific to its training data rather than a genuine, generalizable pattern, and so performs far worse on new data.

Leakage — Any pathway, direct or indirect, by which information from outside the intended training set (often from the future, or from the test set) influences a model's training or a strategy's signal.

In-sample / out-of-sample — In-sample refers to the data used to design, fit, or tune a strategy; out-of-sample refers to data withheld from that process and used only to evaluate the result.

Walk-forward validation — A validation scheme that repeatedly trains on a historical window and tests on the subsequent, not-yet-seen window, rolling both forward through time.

Purged cross-validation — A cross-validation scheme for time-series and overlapping-label data that removes (purges) training observations whose label periods overlap with the test fold, preventing leakage through that overlap.

Embargo period — An additional buffer removed from training data immediately after a test fold, to prevent leakage from residual autocorrelation or slowly-resolving information spilling across the train/test boundary.

Part 1 — Backtesting Architectures: From Signal to Ledger

Before any bias or cost question can be addressed, a backtest needs an architecture: a way of turning historical data into signals, signals into positions, and positions into a simulated profit-and-loss ledger. This part covers the two dominant architectural choices and the mechanics that sit inside either one.

1.1 Vectorised vs. Event-Driven Backtesting

A **vectorised backtest** computes signals, positions, and returns across the entire historical dataset using array operations — a single line like `signal.shift(1) * returns` can compute an entire strategy's daily P&L in one pass. It is fast, simple to write, and ideal for early-stage research and rapid iteration over many parameter choices.

An **event-driven backtest** instead processes one time step (or one discrete market event) at a time, passing it through a pipeline of market-data, signal, order, and fill events — structurally mirroring how a live trading system actually operates. It is slower and more complex to build, but far harder to accidentally corrupt with information that would not have been available at that point in the simulation, because the architecture simply never gives the code access to anything beyond the current event.

Figure 1.1 — Two Backtesting Architectures: Vectorised vs Event-Driven

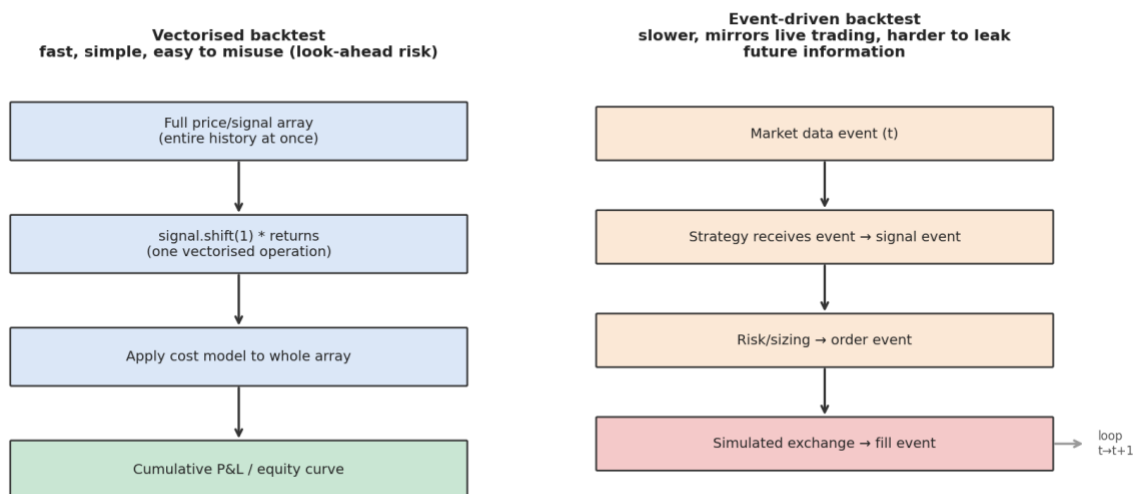


Figure 1.1 — Vectorised backtesting processes the whole history as arrays in a few operations; event-driven backtesting steps through time one event at a time, mirroring a live trading system.

Tip — A common and effective workflow uses both: vectorised backtests for the fast, iterative early research phase, when many signal variants and parameter choices are being screened, followed by an event-driven backtest (or a live paper-trading run) as a final, higher-fidelity check before any strategy is approved — precisely because the two architectures fail differently, and a strategy that survives both is far less likely to be an artifact of either one's blind spots.

```
# Vectorised (fast, but the shift(1) is doing all the bias-safety work)
signal = compute_signal(prices)          # e.g., a moving-average crossover
position = signal.shift(1)                # <-- MUST lag by at least one period
```

```

strategy_returns = position * prices.pct_change()
equity_curve = (1 + strategy_returns).cumprod()

# Event-driven (slower, but structurally cannot see future events)
for event in market_data_stream:
    strategy.on_market_data(event)      # may emit a SignalEvent
    portfolio.on_signal(signal_event)    # may emit an OrderEvent
    exchange.on_order(order_event)      # returns a FillEvent, later
    portfolio.on_fill(fill_event)       # updates cash, positions, NAV

```

1.2 Signal and Position Generation

A raw signal (a momentum score, a z-score, a model's predicted return) is rarely traded directly at face value — it is first transformed into a **position size**, typically through some combination of clipping (capping extreme signal values), scaling (targeting a specific volatility contribution), and normalization (comparing a signal cross-sectionally against the rest of the current universe).

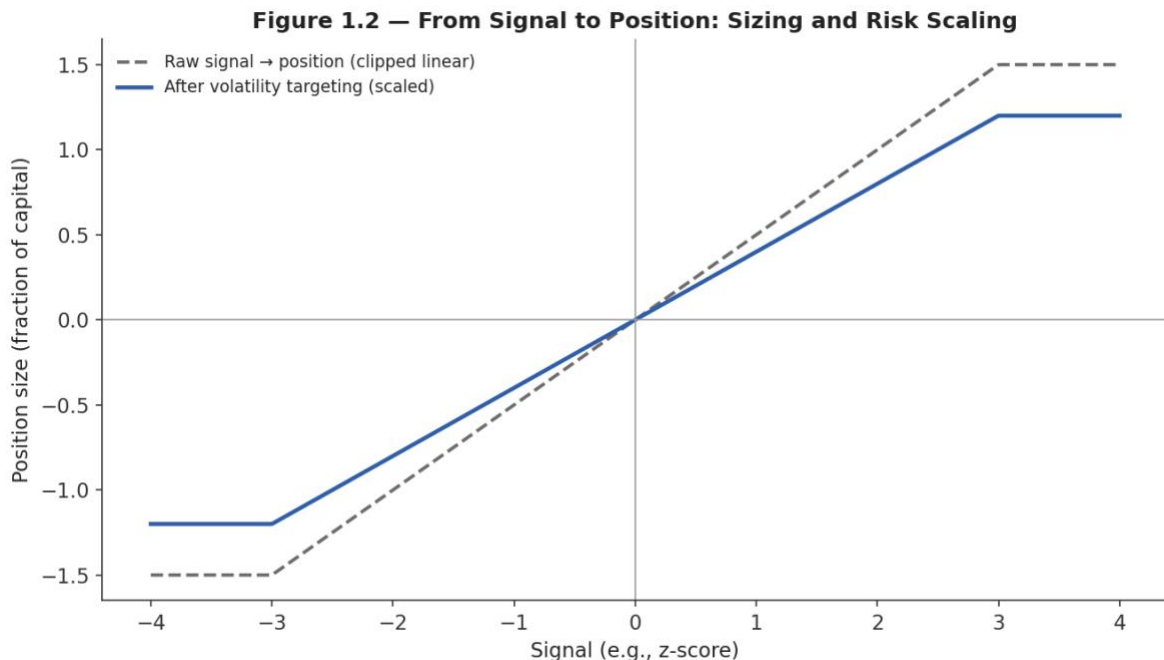


Figure 1.2 — A raw signal is transformed into a position size through clipping and volatility-target scaling; the same z-score can map to very different position sizes depending on these choices.

$$Position(t) = clip(Signal(t), -cap, +cap) \times (target\ volatility / recent\ realized\ volatility)$$

This transformation is not a cosmetic detail — it is where a large fraction of a strategy's realized risk and turnover characteristics are actually determined, and it is also a common, subtle source of look-ahead bias if the 'recent realized volatility' term uses a window that extends past the current point in time (Section 3.1 covers this failure mode directly).

1.3 Portfolio Accounting

Once positions are generated, a backtest needs correct **portfolio accounting**: tracking cash, the market value of long and short positions, and how all three combine into net asset value (NAV) over time, including the effect of any cash generated by short sales, margin requirements, and reinvestment of P&L.

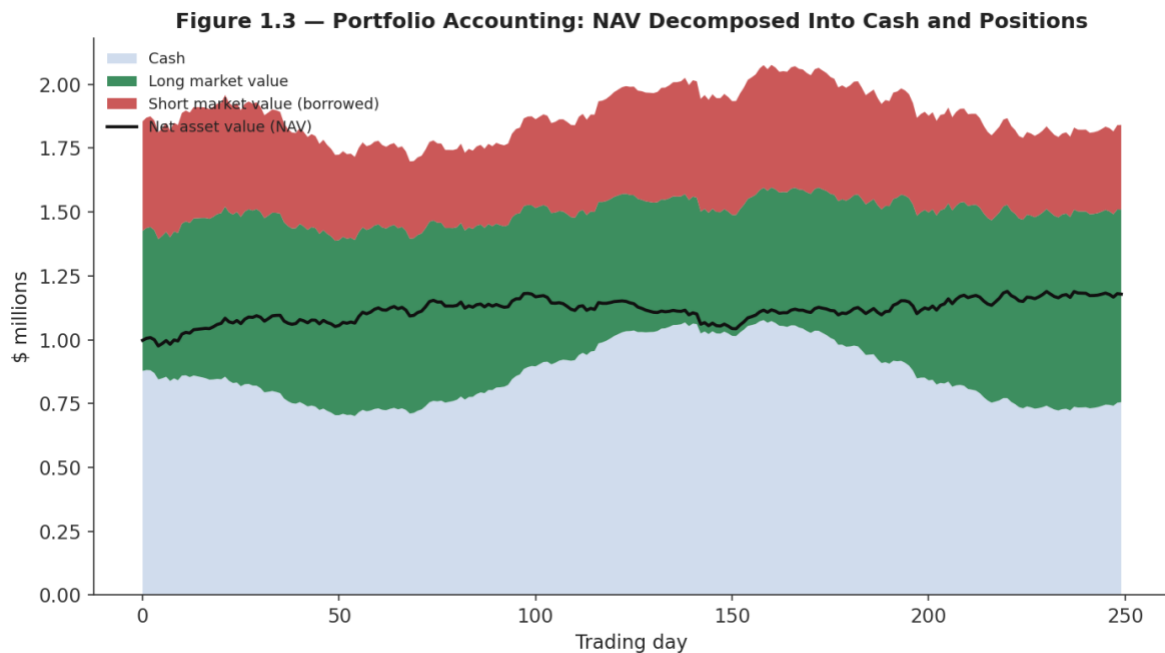


Figure 1.3 — Net asset value decomposed into its cash, long-position, and short-position components over time; correct accounting requires all three to be tracked and reconciled at every step, not just the NAV total.

A surprisingly common bug lives here: a backtest that computes returns correctly for each individual position but fails to correctly account for the cash implications of leverage, margin, or short-sale proceeds will produce a NAV series that is subtly wrong in a way that does not show up until returns are compared against a live-cash-constrained portfolio — another instance of the general theme that backtesting bugs tend to be invisible until they are specifically checked for.

Part 2 — Modelling the Real Cost of Trading

A backtest's gross return — what the strategy would earn if trading were free and instantaneous — is close to meaningless on its own. This part covers each of the frictions that must be modelled to get from a gross return to a number worth trusting.

2.1 Transaction Costs, Slippage, and the Cost Waterfall

Transaction costs and **slippage** are best understood not as a single number but as a waterfall of distinct deductions, each with its own driver and its own sensitivity to how the strategy trades: the bid-ask spread crossed on every trade, the market impact of the trade's own size, explicit fees or commissions, and the financing cost of any leverage or short positions held over time.

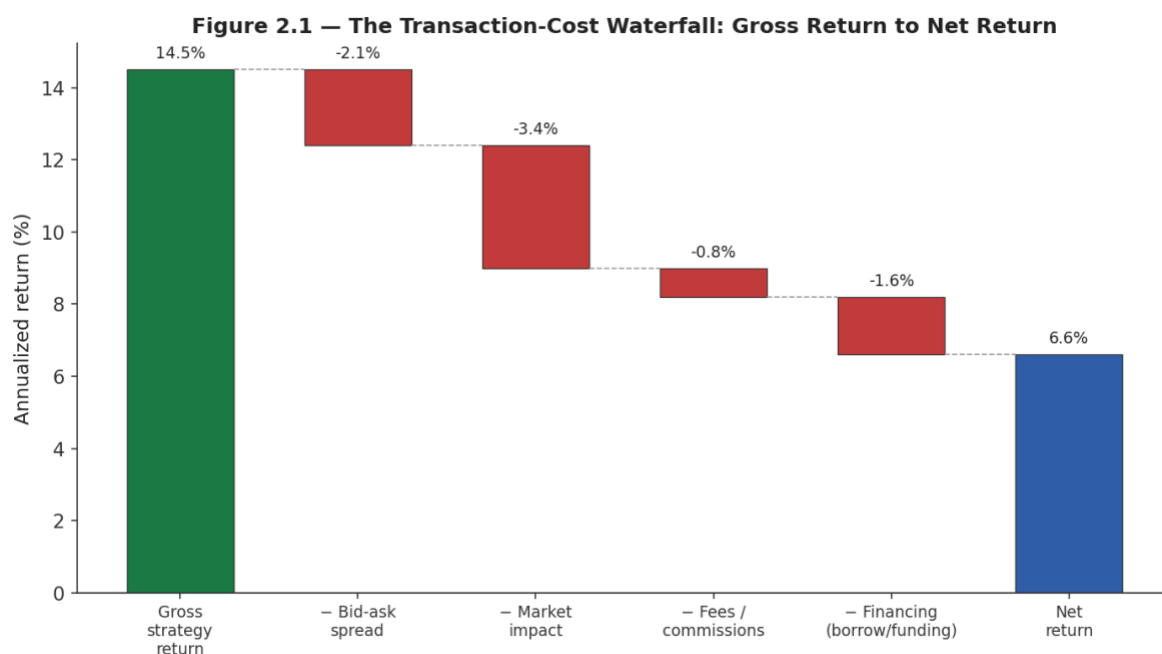


Figure 2.1 — A strategy's gross return is eroded stage by stage: spread, market impact, fees, and financing costs each take a bite before arriving at the number that actually matters — net return.

The purpose of decomposing costs this way, rather than applying a single flat haircut, is that each component scales differently with how the strategy trades: spread cost scales with the number of trades, market impact scales (roughly) with the square root of trade size relative to available liquidity (see the Market Microstructure module), fees scale with trade count and notional, and financing cost scales with how much leverage and short exposure is held and for how long. A strategy redesign that reduces turnover will shrink the spread and fee terms but leave the financing term unchanged — understanding the decomposition tells you which cost a given design choice actually addresses.

2.2 Bid-Ask Spread and Market-Impact Modelling

The single most consequential simplification in a naive backtest is assuming a small, constant transaction cost regardless of trade size, market conditions, or the instrument's typical liquidity. A more realistic model makes cost a function of the size being traded relative to available depth, and of prevailing

volatility — both of which vary over time and, critically, both of which tend to be worst exactly when a strategy most wants to trade (during volatile, fast-moving markets).

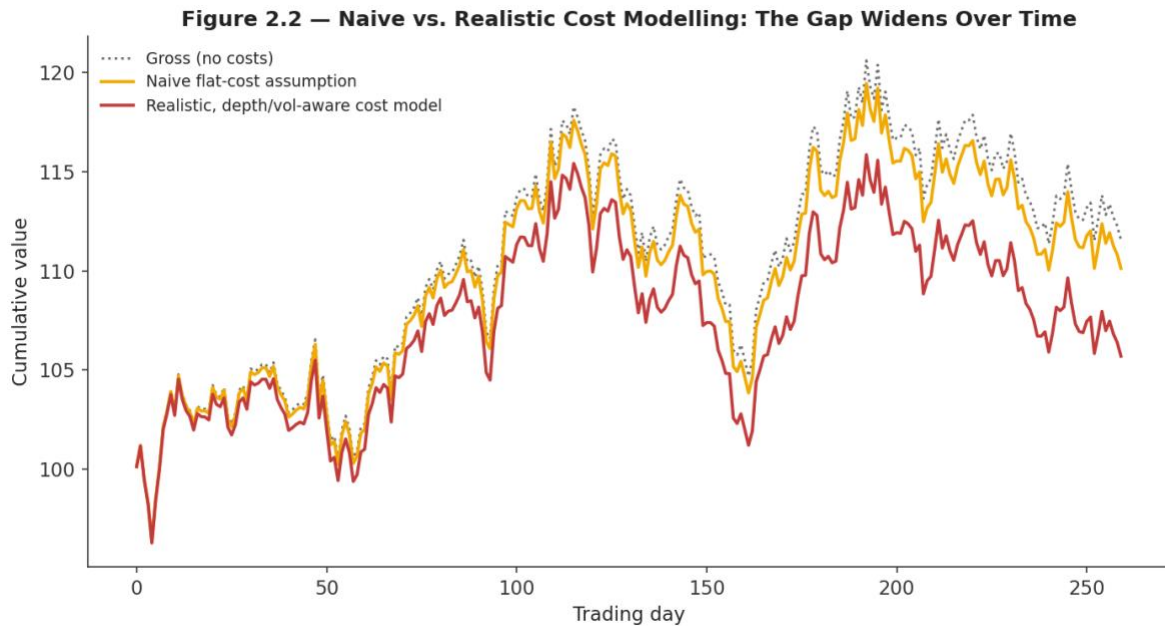


Figure 2.2 — A naive, flat-cost assumption understates the true cost of trading, and understates it more the longer the backtest runs, because it never captures the periods when spreads and impact are genuinely elevated.

Common mistake — Calibrating a cost model once, on calm-market data, and applying it uniformly across the whole backtest period — including periods of market stress when the strategy's signal (and therefore its trading) may be most active. A cost model that does not vary with volatility will systematically understate costs exactly when it matters most.

2.3 Latency, Funding, and Borrow Costs

Latency — modelled in a backtest as a delay between signal generation and simulated execution — matters most for strategies operating on short horizons, where the price used for the fill should reflect what was actually achievable some interval after the signal fired, not the price at the exact moment the signal was computed.

Borrow costs (paid to hold a short position) and **funding costs** (paid or received on perpetual futures and similar instruments) are frequently modelled as a small, constant annualized rate — a simplification that can be badly wrong for specific securities that become 'hard to borrow' (where fees can spike from a few percent to well over 50% annualized) or during funding-rate regime shifts in crypto perpetual markets.

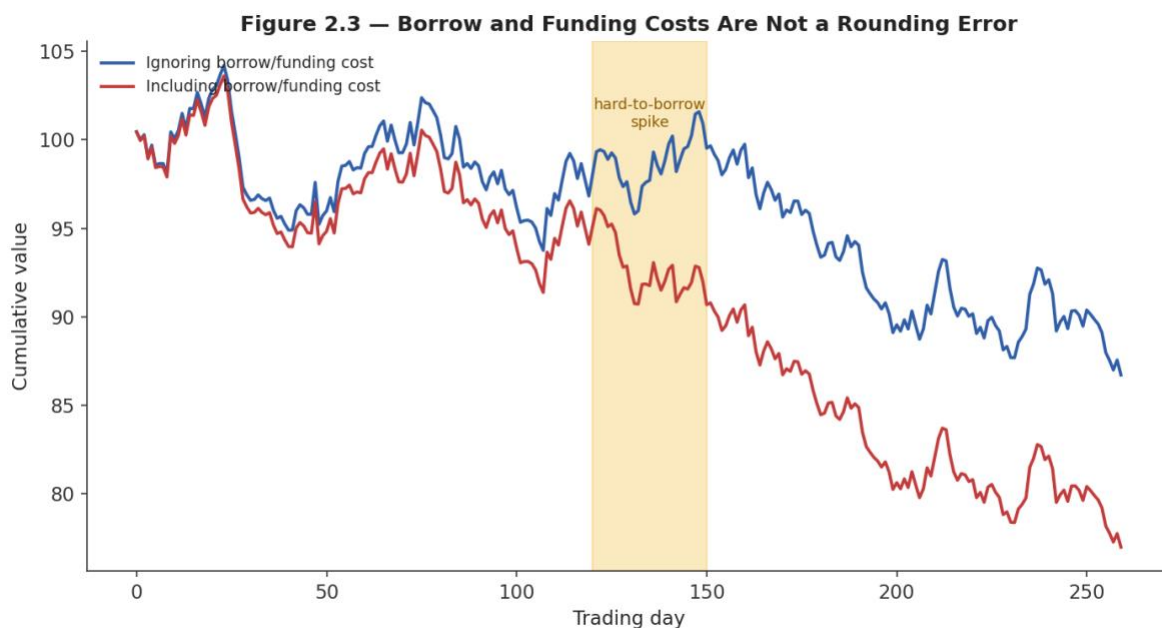


Figure 2.3 — Ignoring borrow and funding costs, or modelling them as a small constant, misses episodes — like a hard-to-borrow spike — that can materially change a short-heavy strategy's realized return.

$$\text{Daily borrow cost} = \text{Short market value} \times (\text{annualized borrow rate} / 252)$$

2.4 Partial Fills

A backtest that assumes every order fills completely at the assumed price implicitly assumes infinite available liquidity — a version of the same mistake covered in the Market Microstructure module's market-depth discussion. **Partial fills** occur when an order's size exceeds what resting liquidity can absorb at an acceptable price, and a realistic backtest needs to model the resulting fill ratio as a function of order size relative to typical market depth.

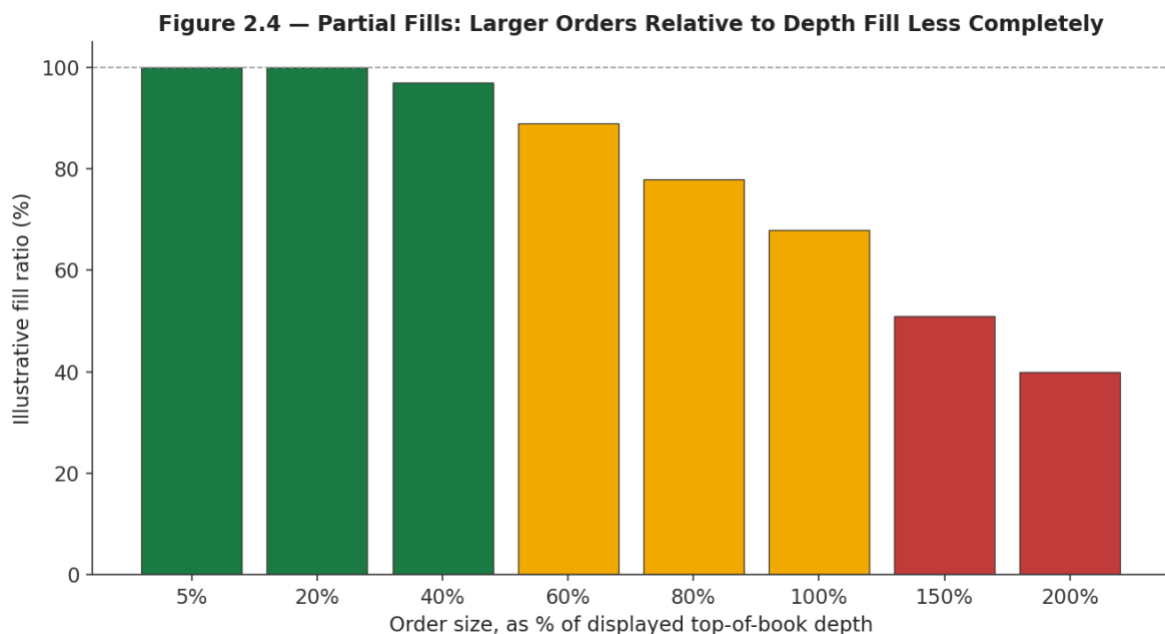


Figure 2.4 — As order size grows relative to displayed market depth, the fraction of the order that actually fills (at or near the intended price) falls — a large order in a thin market may only be partially executable within the assumed cost and time budget.

Modelling partial fills correctly matters most for strategies with limited capacity or strategies trading less liquid instruments: a backtest that always assumes full fills will overstate both the achievable position size and the achievable return, in a way that only becomes visible once the strategy is actually deployed with real capital against real, finite liquidity.

Part 3 — The Biases That Lie to You

This part covers the four biases most responsible for backtests that look far better than any live strategy could actually perform — each illustrated with a quantitative example of exactly how much damage it can do.

3.1 Look-Ahead Bias

Look-ahead bias occurs when a signal or decision at time t uses information that would not actually have been available at time t — a single-day (or even single-bar) error that is remarkably easy to introduce accidentally and remarkably difficult to notice by inspection, because the resulting backtest code often looks completely reasonable.

Figure 3.1 — Look-Ahead Bias: A One-Day Bug Can More Than Triple the Apparent S
2.85

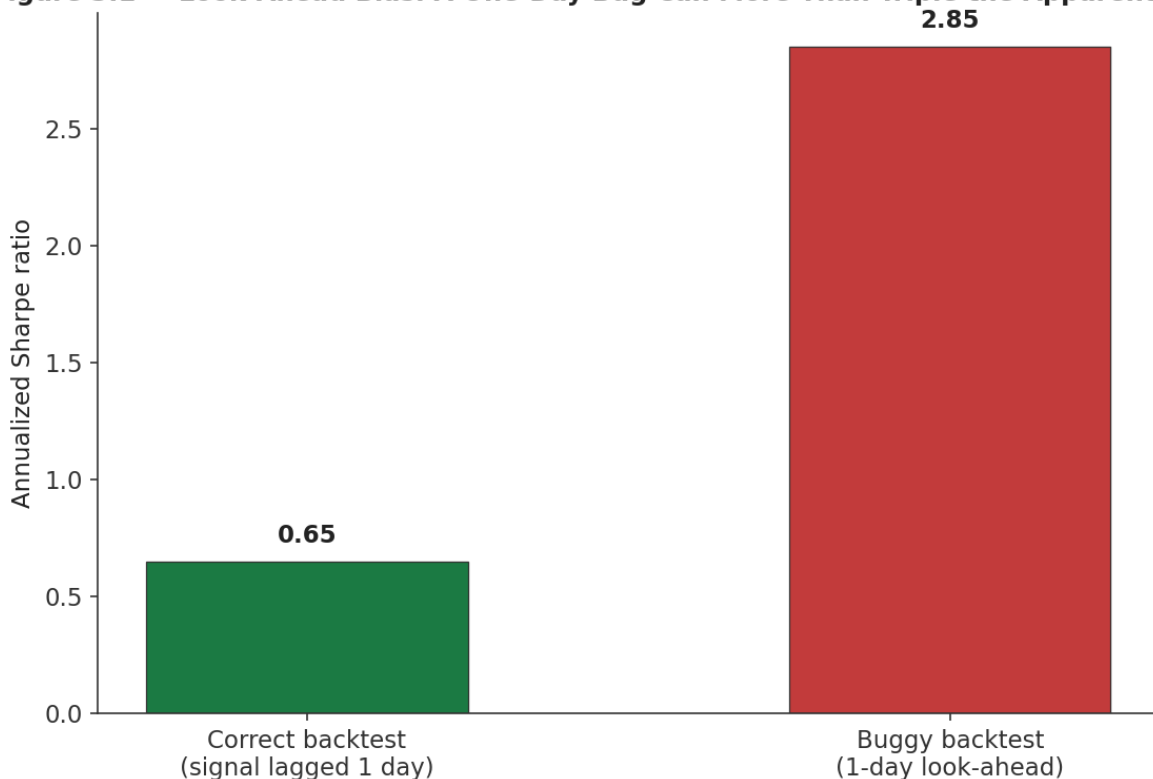


Figure 3.1 — A backtest with a one-day look-ahead bug (using today's close to generate a signal traded at today's open, say) can inflate the apparent Sharpe ratio by several times over the correctly-lagged version.

Worked example — A momentum signal is defined as today's close divided by yesterday's close. The (buggy) backtest computes this signal using today's close and trades it at today's close — implicitly assuming the trade could be placed using information (today's close) that would only be known after the market has already closed for the day. Lagging the signal by one full trading day before use — trading tomorrow based on today's already-known signal — is the fix; in the illustrative example here, the bug alone inflates the annualized Sharpe ratio from roughly 0.65 to roughly 2.85.

```
# Buggy: uses today's return to trade today (look-ahead)
```

```

signal = prices.pct_change()
position = signal                                     # WRONG: trades today using today's info
strategy_returns = position * prices.pct_change()

# Correct: lag the signal by (at least) one period before trading on it
position = signal.shift(1)                             # today's position uses YESTERDAY's signal
strategy_returns = position * prices.pct_change()

```

3.2 Survivorship Bias

Survivorship bias occurs when the historical universe used in a backtest is implicitly restricted to instruments that still exist today — silently excluding every company that was delisted, went bankrupt, or was acquired at a distressed price along the way. Because failure is not random with respect to future returns (failing companies almost by definition underperformed before failing), this bias is directional and systematic, not merely noisy.

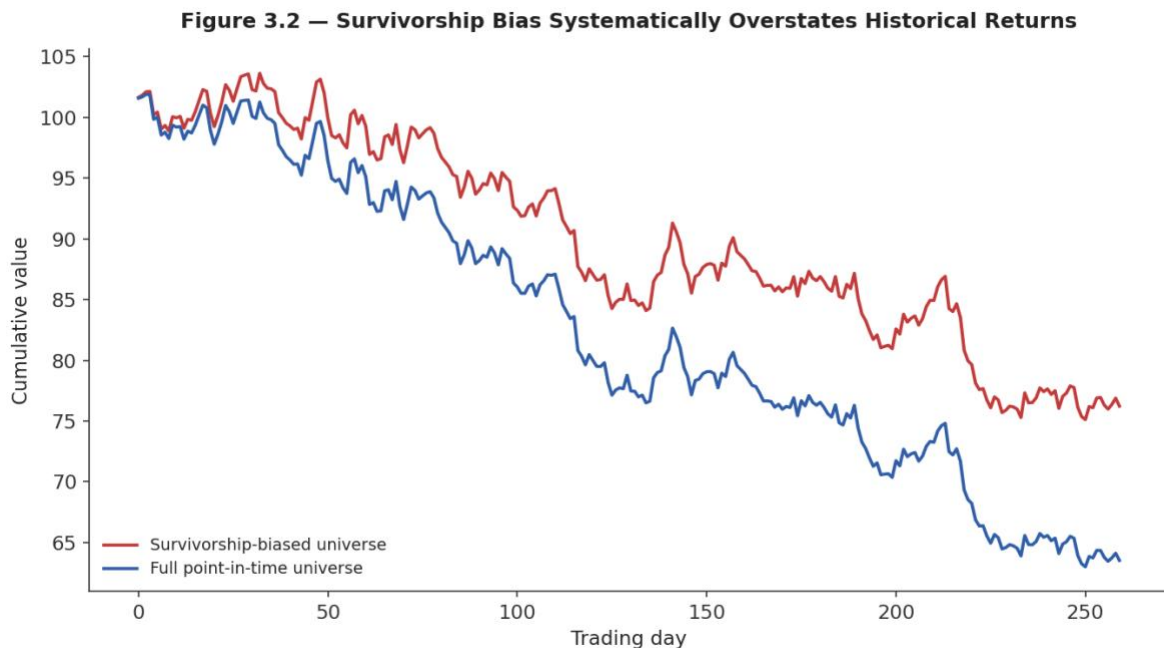


Figure 3.2 — A backtest run only on today's surviving universe of companies (red) overstates historical returns relative to a correctly point-in-time universe that includes firms later delisted or failed (blue).

The fix requires a **point-in-time universe**: a dataset that records, for every historical date, exactly which instruments were tradeable as of that date, including ones that no longer exist — which is a data-engineering requirement, not merely a modelling choice, and one reason survivorship bias is so persistent in practice: many convenient, freely available datasets simply do not include this information, and building or purchasing a proper point-in-time dataset is a real cost that a naive backtest can appear to avoid.

3.3 Selection Bias and Data Snooping

Selection bias occurs when the sample of data, instruments, or time periods studied is not representative of what the strategy will actually trade on — testing a strategy only on large-cap

technology stocks and then deploying it broadly, for instance. **Data snooping** is a closely related, often more insidious problem: repeatedly testing variations of a strategy against the same dataset until one appears to work, which inflates the apparent statistical significance of whatever was finally kept.

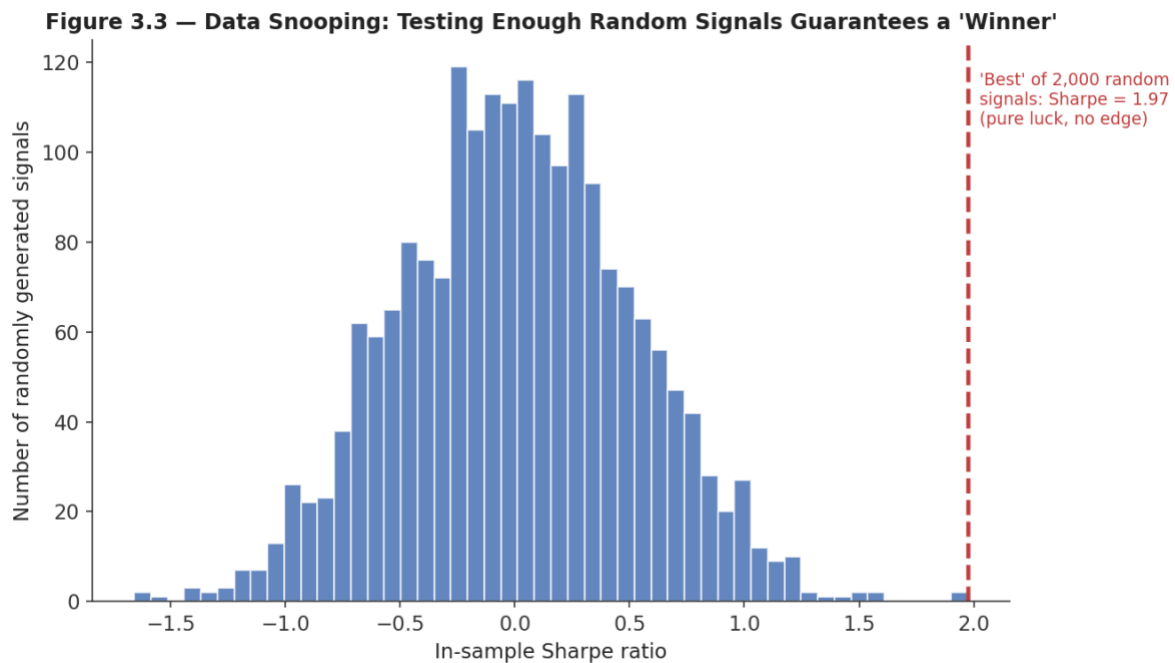


Figure 3.3 — Testing thousands of purely random trading signals against the same historical data guarantees that some of them will show an impressive-looking Sharpe ratio by chance alone, with zero genuine edge.

The connection to the Statistics prerequisite is direct: evaluating one pre-specified hypothesis against data is standard statistical inference; evaluating the best of many hypotheses against the same data, without correcting for how many were tried, systematically overstates significance — the same multiple-testing problem that appears in medical research and genomics, applied to strategy research. A Sharpe ratio of 2.0 found after testing 5 strategies is meaningful; the same Sharpe ratio found after testing 5,000 variations is close to guaranteed by chance and should be treated with corresponding skepticism until validated out-of-sample.

3.4 Overfitting and Leakage

Overfitting occurs when a model or rule-based strategy has fit patterns specific to its training data — including pure noise — rather than a genuine, generalizable relationship, and so performs markedly worse on new data than its in-sample performance would suggest. **Leakage** is the broader category of any pathway by which information outside the intended training set influences training or signal construction, of which look-ahead bias (Section 3.1) is one specific, temporal instance.

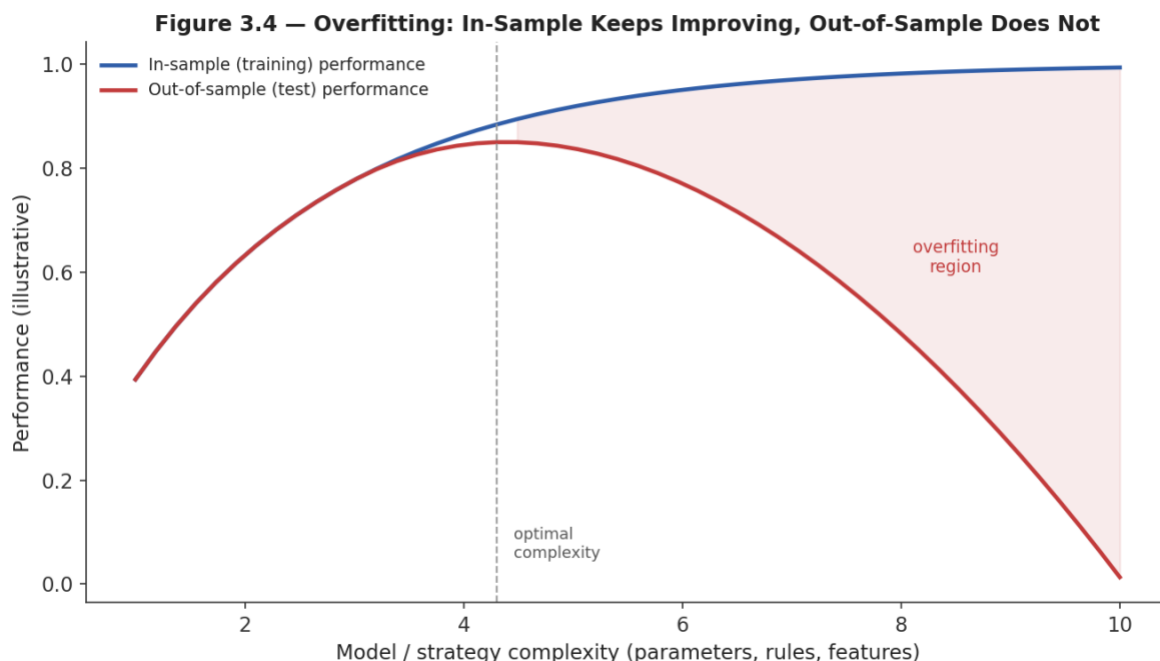


Figure 3.4 — As model or strategy complexity increases, in-sample performance keeps improving, but out-of-sample performance peaks and then deteriorates — the gap between the two curves is the overfitting the strategy has accumulated.

Overfitting risk scales directly with the number of free parameters or rules relative to the amount of independent data available to fit them — a strategy with twenty tunable parameters fit against three years of daily data has vastly more freedom to fit noise than a strategy with two parameters fit against the same data, and correspondingly needs much more rigorous out-of-sample validation (Part 4) before its in-sample performance can be trusted at all.

Tip — A useful heuristic: if you cannot explain, in one sentence and without reference to the backtest results, why a specific parameter or rule should improve the strategy, that parameter or rule is a candidate for overfitting rather than genuine signal — this is the same 'economic rationale first' discipline from the Alpha Research module, applied at the level of individual model choices rather than entire strategy families.

Part 4 — Validation Methodology

Understanding the biases in Part 3 motivates the validation methodology in this part: a set of procedures specifically designed to detect overfitting and leakage before a strategy is trusted with real capital.

4.1 In-Sample vs. Out-of-Sample

The most basic validation discipline is holding out a portion of the data — the **out-of-sample** period — that is never used, in any way, during strategy design, parameter selection, or model fitting. The **in-sample** period is where all the exploratory work happens; the out-of-sample period is examined exactly once, after every design decision has already been locked in, to get an honest estimate of how the strategy will perform on data it has never influenced.

Data → [In-sample: design, fit, tune] → [Out-of-sample: evaluate once, do not iterate further]

Common mistake — Treating out-of-sample data as a second in-sample period — looking at out-of-sample performance, noticing it is disappointing, tweaking the strategy, and re-checking the same out-of-sample period. Each additional look re-introduces the data-snooping problem from Section 3.3 and destroys the very property (never having influenced the design) that made the out-of-sample check meaningful in the first place.

4.2 Walk-Forward Validation

A single in-sample/out-of-sample split uses only one out-of-sample period, which can itself be an unrepresentative sample of market conditions. **Walk-forward validation** addresses this by repeating the split multiple times: train on a window, test on the subsequent window, then roll both windows forward in time and repeat, producing a sequence of genuinely out-of-sample test periods that collectively span a much larger fraction of the available history.

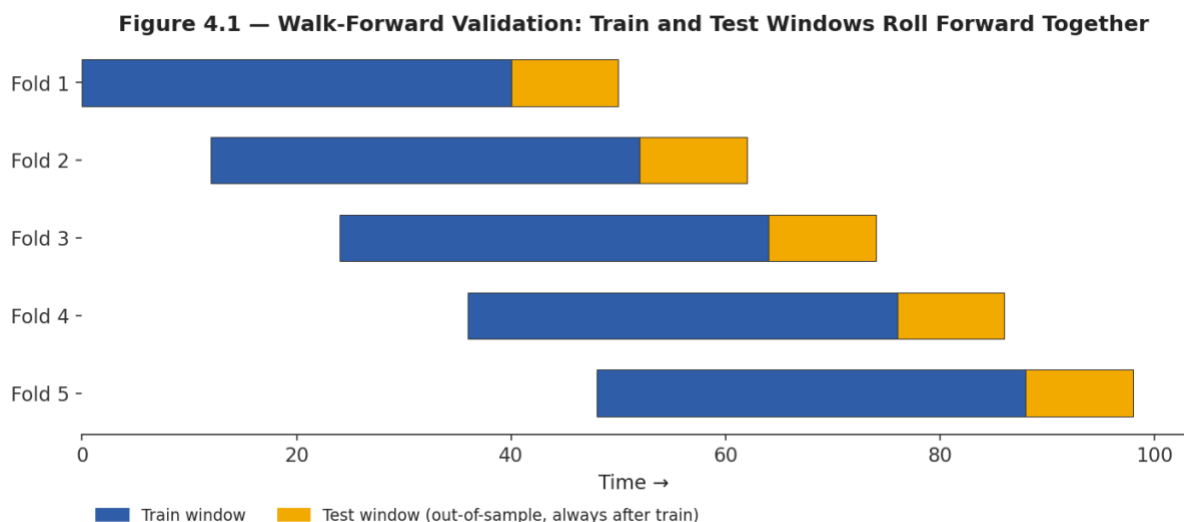


Figure 4.1 — Walk-forward validation: each fold trains on a historical window and tests on the period immediately following it, then both windows roll forward — every test period is out-of-sample relative to its own training window.

This is the correct generalization of train/test splitting to time-series data, and it directly addresses the first common beginner mistake called out in this module: applying standard k-fold cross-validation — which shuffles data randomly across folds — to time-series data is a serious error, because it allows a model to be 'trained' on data from after the period it is being tested on, silently reintroducing look-ahead bias through the back door of the validation procedure itself.

4.3 Purged Cross-Validation and Embargo Periods

Even walk-forward validation is not automatically sufficient when a strategy's features or labels are constructed from overlapping windows of data (a very common situation — e.g., a 20-day rolling feature, or a label defined by a 5-day forward return). In that case, a training observation whose window overlaps with the test period can leak information into the model even though its timestamp is technically 'before' the test fold begins.

Figure 4.2 — Purged Cross-Validation: Gaps Around the Test Fold Prevent Leakage

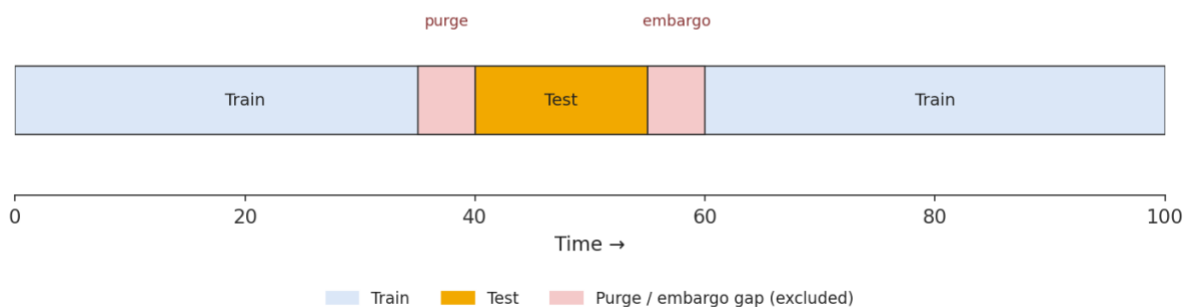


Figure 4.2 — Purged cross-validation removes training observations whose label or feature windows overlap the test fold (the purge), and adds a further embargo period immediately after the test fold to guard against residual, slowly-decaying information leakage.

Purge: drop training samples whose window overlaps $[test_start, test_end]$ Embargo: also drop training samples in $[test_end, test_end + embargo_length]$

This technique, developed prominently in the context of machine-learning-based financial signals, is now considered standard practice for any ML-based strategy whose features or labels involve overlapping time windows — which describes a large majority of practically useful financial ML features — and its absence is one of the most common, and most damaging, methodological gaps in ML-based strategy research.

```
# Purged + embargoed walk-forward split (schematic)
def purged_split(index, test_start, test_end, purge, embargo):
    train_mask = (index < test_start - purge) | (index > test_end + embargo)
    test_mask = (index >= test_start) & (index <= test_end)
    return index[train_mask], index[test_mask]

for test_start, test_end in walk_forward_windows(index):
    train_idx, test_idx = purged_split(index, test_start, test_end,
                                      purge=label_horizon, embargo=label_horizon)
    model.fit(X[train_idx], y[train_idx])
    score = evaluate(model, X[test_idx], y[test_idx])
```

Practical Exercise

Take a working strategy and deliberately introduce a one-day look-ahead bug; show quantitatively how much it inflates the Sharpe ratio, then fix it.

Suggested approach

- Start from a simple, correctly-lagged strategy (e.g., a moving-average crossover) with a known, modest Sharpe ratio.
- Introduce the bug: remove the `.shift(1)` (or equivalent lag) so that the signal computed using bar t 's data is traded as of bar t , not bar $t+1$.
- Re-run the backtest and record the new Sharpe ratio, maximum drawdown, and win rate alongside the original, correctly-lagged figures.
- Quantify the inflation: report the buggy-to-correct ratio for each metric, and inspect a handful of individual trades to see concretely which ones only 'worked' because of the leaked information.
- Fix the bug and confirm the corrected backtest reproduces the original, honest performance.

```
# Practical exercise skeleton
def backtest(prices, lag):
    signal = compute_signal(prices)
    position = signal.shift(lag)      # lag=1 correct, lag=0 introduces look-ahead
    returns = position * prices.pct_change()
    return returns.dropna()

correct = backtest(prices, lag=1)
buggy   = backtest(prices, lag=0)

print('Correct Sharpe:', sharpe_ratio(correct))
print('Buggy Sharpe:  ', sharpe_ratio(buggy))
print('Inflation factor:', sharpe_ratio(buggy) / sharpe_ratio(correct))
```

Common Beginner Mistakes

- **Using standard k-fold cross-validation on time-series data** — this shuffles observations randomly across folds, which both breaks temporal ordering (allowing training on 'future' data relative to a test point) and, for overlapping-window features, reintroduces leakage even when timestamps are respected; walk-forward or purged/embargoed cross-validation must be used instead.
- **Backtesting on a survivorship-biased universe** — e.g., only stocks that still exist today, silently excluding every historical failure and systematically overstating returns.
- Treating a single in-sample/out-of-sample split as sufficient validation for a strategy with many tunable parameters.
- Applying a flat, constant transaction-cost assumption regardless of trade size, instrument liquidity, or market conditions.
- Re-examining and iterating against the same out-of-sample or test data more than once, which quietly converts it into another in-sample dataset.

Advanced Extensions

The following are Level 4 topics, introduced here only by name so you know what to look for when you reach them:

- **Purged and embargoed cross-validation for ML-based signals** — the full treatment of designing leakage-safe validation schemes for machine-learning models with overlapping labels, including nested cross-validation for hyperparameter selection.
- **Multi-strategy backtest infrastructure** — building a single backtesting engine capable of correctly simulating and accounting for many strategies trading concurrently, sharing capital, risk limits, and execution capacity.

Professional Applications

- **Strategy validation before capital allocation** — every strategy proposed for live trading passes through the bias checks and validation methodology in this module as a formal gate, typically owned by an independent research-review or risk function rather than the strategy's own author.
- **Ongoing model monitoring in production** — the same walk-forward logic used to validate a strategy before launch is reapplied continuously after launch, comparing live, realized performance against what the validation methodology predicted, to catch decay or a live-vs-backtest discrepancy as early as possible.

Where to Go Next

This module covered the architecture, cost modelling, bias controls, and validation methodology that every strategy must pass through before being trusted. From here, the curriculum's Risk Metrics module covers how a validated strategy's risk is measured and sized, and the Level 4 Advanced Projects in Part VI cover building the production-grade, multi-strategy backtesting infrastructure this module's concepts are ultimately implemented inside of.